Introduction to Software Systems
COMP 206

Instructor: Jacob Errington and Max Kopinsky
School of Computer Science
McGill University

Lesson 5 : Text stream processing with pipe and Shell wild cards

Text stream processing commands

- Text stream processing commands process text from a file or standard input line by line.

- `$ cut -d <delimiter> [file]`
  - `cut` command is used to extract specific sections of text from each line of a file or standard input. It works well with structured text (like columns or delimited data). You can select output by `-f` to extract fields separated by a delimiter
  - Example: `cut -d ',' -f 1,3 data.csv`

- `$ wc [-clw] [file]`
  - `wc` (word count) simply counts the number of words, lines, and characters in the file(s).
  - The three options, clw, stand for character, line, and word respectively, and tell `wc` which of the three to count.
  - Example: `wc -l data.csv`

Text stream processing commands

- `$ sort [options] [file]`
  - `sort` command is used to sort data in file or standard input (in case of the absence of the file argument)
  - `-n` option treats sorting key as numeric (Important in ensuring 5 comes before 25, etc.)
  - `-r` sorts in descending order
  - `-o outputfile` writes the sort output to output file. Otherwise, sort will write to the standard output.

- `$ uniq [file]`
  - `uniq` (unique) command filters out repeated lines from a file or input stream. It removes consecutive duplicate lines and prints only one instance of each.
  - `-c` option prefix each line with the number of occurrences
  - **`Important:`** uniq only works properly on sorted input, because it only compares adjacent lines.

Text stream processing commands

- Text stream processing commands are often chained together with pipes (|) to build powerful text-processing workflows.

- Example:

```
$cat data.txt | cut -f2 | sort | uniq -c | sort -nr
```

Shell Wild Cards

Wild cards

- A wildcard is a symbol used to replace one or more characters in a filename. In Bash, these are:

- The “asterisk”: Any pattern

  `*.doc`

- The “question mark”: Any single character

  ?`at.doc`

- The “square brackets”: Any character within the brackets
- `cat.d[aoz]c`

Wild cards

- The “square brackets”: Any character within the brackets
- `cat.d[aoz]c`

Inside brackets…

- Two characters separated by hyphen are a range,

  for example [a-d] is equivalent to [abcd]

- You can put ! at the front to match anything not in the square brackets, for example, [!a] matches any single character

Wild cards - Example

*	any pattern
?	any single char
[ ] or	

Which files matches with the wildcards below?

- `*.doc`
- `?at.doc`
- `cat.d[aoz]c`
- `*.d[aoz]?`

john.doc

cat.doc

bat.doc

cat.dac

file.dat

cat.dpt

file.txt

Wild cards - Example

*	any pattern
?	any single char
[ ] or	

Which files matches with the wildcards below?

- `*.doc`
- `?at.doc`
- `cat.d[aoz]c`
- `*.d[aoz]?`

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Wild cards - Example

*	any pattern
?	any single char
[ ] or	

Which files matches with the wildcards below?

- `*.doc`
- `?at.doc`
- `cat.d[aoz]c`
- `*.d[aoz]?`

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Wild cards - Example

*	any pattern
?	any single char
[ ] or	

Which files matches with the wildcards below?

- `*.doc`
- `?at.doc`
- `cat.d[aoz]c`
- `*.d[aoz]?`

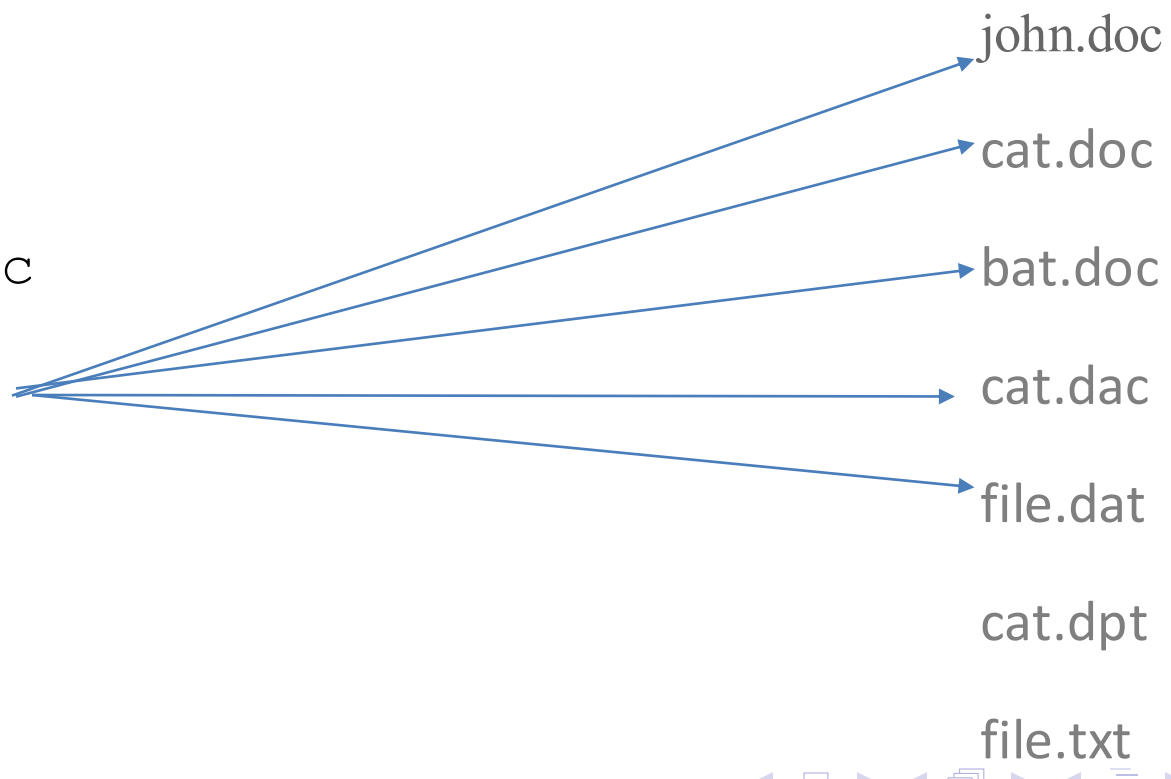
john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Wild cards - Example

*	any pattern
?	any single char
[ ] or	

Which files matches with the wildcards below?

- `*.doc`
- `?at.doc`
- `cat.d[aoz]c`
- `*.d[aoz]?`



Filename expansion

- The shell expands the wildcard patterns before giving them as arguments. For example, if you type `ls *.doc,` the shell expands it into

  `ls john.doc cat.doc bat.doc`

- the command has no idea that you used wildcards, the shell is the one doing the work

- You can see this by yourself with the echo command: Try `echo ls *.doc` in a directory with doc files.

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt

Filename expansion

- Note that if the wildcards can't be matched, the shell itself won't do anything with them or send an error message, and the wildcards will simply be sent to the command literally
  - For example, in a directory with no .doc files, `echo ls *.doc` will echo back `ls *.doc` and `ls` will think that you're trying to open a file literally called `*.doc`

- Note that if a filename starts with a dot (i.e. is hidden), it must be matched explicitly
  - For example, `*.txt` does not match `.hidden.txt`

john.doc
cat.doc
bat.doc
cat.dac
file.dat
cat.dpt
file.txt
.hidden.txt

Filename expansion

- Filename expansion is applied on each segments of a pathname separately (i.e each parts separated by/).
  - For example, if you are in the comp206 directory below and want to match Docs/myfile.txt, typing `*.txt` would not match it. However, `Docs/*.txt` or `*/*.txt` would match it.

```
comp206/
        foo.txt
        bar.txt
        Docs/
            myfile.txt
```

grep

- `grep` is used to search for the patterns in files.
- `grep [options] STRING FILE_LIST`
- STRING above is the pattern to match, and it is called a regular expression (or regex). We are not covering regex in this course, but here is a great resource if you would like to learn more about it: https://regexone.com/

- Some common options include:
  - -i : ignore case
  - -n : display the line number along with the line on which a match was found
  - -c : report only a count of the number of lines containing matches
  - -v : invert the search, displaying only lines that do not match
  - -l : list filenames, but not lines, in which matches were found

Example using grep

- Consider the following content of `students.txt` file :

```
Alex
Marc
Micheal
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem
```

```
$ grep 'Je' students.txt
Jeremy
Jessica
Jean-Sebastien
```

Example using grep

- Consider the following content of `students.txt` file :

```
Alex
Marc
Micheal
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem
```

```
$ grep -n 'Je' students.txt
6:Jeremy
7:Jessica
10:Jean-Sebastien
```

Example using grep

- Consider the following content of `students.txt` file :

```
Alex
Marc
Micheal
Ting
Juan
Jeremy
Jessica
Yannick
Nicolas
Jean-Sebastien
Nadeem
```

```
$ grep -c 'Je' students.txt
3
```